TRƯỜNG ĐẠI HỌC CÔNG NGHIỆP TP. HỒ CHÍ MINH

SỔ TAY THUYẾT TRÌNH

Báo cáo cuối kỳ môn Phân tích dữ liệu lớn

Đề tài 8

Phân tích dữ liệu IoT và bảo trì dự đoán

Giảng viên hướng dẫn:	TS. Võ Thị Thanh Vân
Học viên thực hiện:	Phan Nguyễn Mai Phương
Mã số học viên:	25002711
Ngành:	Khoa học máy tính

Mỗi slide một trang: hình slide, nhìn vào đâu, đọc gì, khi nào chuyển.

Tháng 10 năm 2026

Contents

Cách dùng sổ tay **2**

Trước buổi báo cáo **3**

Phần nói theo từng slide **4**

Slide 01 – Phân tích dữ liệu IoT và bảo trì dự đoán . . . 5
Slide 02 – Bài toán: xác định thời điểm cần bảo trì cho từng động cơ . . . 6
Slide 03 – Quy trình thực hiện gồm năm bước . . . 7
Slide 04 – Nguồn dữ liệu: bộ NASA C-MAPSS gồm bốn tập con . . . 8
Slide 05 – Mỗi bản ghi là một cycle của một động cơ, gồm 26 trường . . . 9
Slide 06 – Dữ liệu đầy đủ; tuổi thọ động cơ từ 128 đến 362 cycle . . . 10
Slide 07 – Dữ liệu được tổ chức theo ba lớp Bronze, Silver, Gold . . . 11
Slide 08 – Lưu trữ và xử lý trên cụm Hadoop: HDFS, MapReduce, YARN . . . 12
Slide 09 – PySpark cho kết quả trùng khớp với Pandas . . . 13
Slide 10 – Truy vấn dữ liệu bằng SQL trên SQLite và Spark SQL . . . 14
Slide 11 – Sáu bước biến đổi từ file text thành 17.731 mẫu huấn luyện . . . 15
Slide 12 – Khi dữ liệu tăng 100 lần, Spark tính nhanh hơn Pandas 5,5 lần . . . 16
Slide 13 – Làm mượt EWMA giảm khoảng 86% dao động ngắn hạn . . . 17
Slide 14 – 12 sensor thay đổi cùng chiều ở cả 100 động cơ khi gần hỏng . . . 18
Slide 15 – Health Index phát hiện bất thường ở cả 100 động cơ train . . . 19
Slide 16 – Mô hình HistGradientBoosting dự đoán RUL với MAE 15,35 cycle . . . 20
Slide 17 – Cảnh báo sớm: ngưỡng 40 cycle nâng tỷ lệ báo kịp thời lên 94% . . . 21
Slide 18 – Lịch bảo trì xác định đúng 21/21 động cơ cần bảo trì ngay . . . 22

Demo dashboard **23**

Slide 19 – Kết luận, hạn chế và hướng phát triển . . . 29
Slide 20 – Trân trọng cảm ơn quý thầy cô đã lắng nghe . . . 30

Câu hỏi phản biện **31**

Số liệu tra nhanh **33**

Cách dùng sổ tay

Mỗi trang ứng với một slide, theo đúng thứ tự trên máy chiếu.

- **Ô vàng ở góc phải** là thời lượng nên dành cho slide, kèm mốc giờ khi kết thúc slide. Nếu đồng hồ chạy quá mốc 1 phút, đọc gọn lại.
- **Khung vàng "Nhìn vào"** chỉ chỗ cần đưa mắt hoặc chỉ tay trên slide.
- **Phần "Đọc"** là lời nói nguyên văn, có ví dụ cụ thể để người không học chuyên ngành cũng theo được. Số in đậm là số quan trọng, nói chậm và rõ ở các số này.
- **Khung xám "Nói thêm"** là chi tiết sâu hơn. Chỉ đọc khi còn dư thời gian, hoặc dùng để trả lời khi bị hỏi.
- **Dòng "Chuyển slide"** ở cuối trang là câu nối sang slide tiếp theo.

Demo dashboard thực hiện ngay sau slide 18, trước hai slide kết luận và cảm ơn. Cả 20 slide đều là nội dung chính. Cuối sổ tay có câu hỏi phản biện và số liệu tra nhanh.

Kế hoạch thời gian

Phần	Slide	Hết phần lúc
Giới thiệu bài toán và quy trình	1–3	1:25
Dữ liệu và hạ tầng	4–12	8:05
Phân tích, dự đoán và lịch bảo trì	13–18	12:15
Demo dashboard	Sau 18	14:15
Kết luận	19	14:50
Cảm ơn	20	15:00

Lời nói và chuyển ý dự kiến 13 phút; demo 2 phút. Các mốc cộng dồn đã tính demo. Khung "Nói thêm" không thuộc thời lượng chính. Tập đọc có bấm giờ để giữ tổng thời gian khoảng 15–16 phút; nếu chậm, rút chi tiết phụ và thao tác chuyển trang, không bỏ phần dữ liệu và hạ tầng ở slide 4–12.

Trước buổi báo cáo

Ngày hôm trước

- Mở thử `slides/cmapss_fd001_presentation.pptx` trên máy sẽ dùng để chiếu, kiểm tra font Arial hiển thị đúng tiếng Việt.
- Chép thêm bản PDF `slides/cmapss_fd001_presentation.pdf` vào USB, phòng khi máy trình chiếu không có PowerPoint.
- Chạy thử dashboard một lần theo lệnh bên dưới, đi qua đủ 5 bước demo.
- Tập giải thích khóa dữ liệu, window 30 cycle và hai mức cảnh báo bằng các ví dụ trong sổ tay.
- Đọc to sổ tay một lượt có bấm giờ.

15 phút trước khi lên

1. Mở terminal ở thư mục gốc project và chạy:
   `uv run --frozen --extra dashboard streamlit run app.py`
2. Đợi trình duyệt mở `http://localhost:8501`, để sẵn ở trang **Tổng quan**. Nhấn `Ctrl` và phím `+` hai lần để chữ to hơn.
3. Mở PowerPoint ở chế độ Presenter View (Slide Show → Use Presenter View), để mình thấy speaker notes còn khán giả chỉ thấy slide.
4. Để sẵn hai cửa sổ: PowerPoint và trình duyệt. Chuyển qua lại bằng `Alt+Tab`.

Nếu có sự cố

- **Dashboard không mở được:** bỏ qua demo, nói "Em xin trình bày kết quả demo qua slide" rồi quay lại slide 15, 17, 18.
- **Máy không có PowerPoint:** chiếu bản PDF; speaker notes không còn, nên đọc theo sổ tay này.
- **Quên số liệu:** lật trang "Số liệu tra nhanh" ở cuối sổ tay.

Phần nói theo từng slide

Slide 01/20 Phân tích dữ liệu IoT và bảo trì dự đoán

20 giây hết slide lúc 0:20



Nhìn vào
Chỉ tên đề tài, môn học và tên giảng viên hướng dẫn trên bìa.

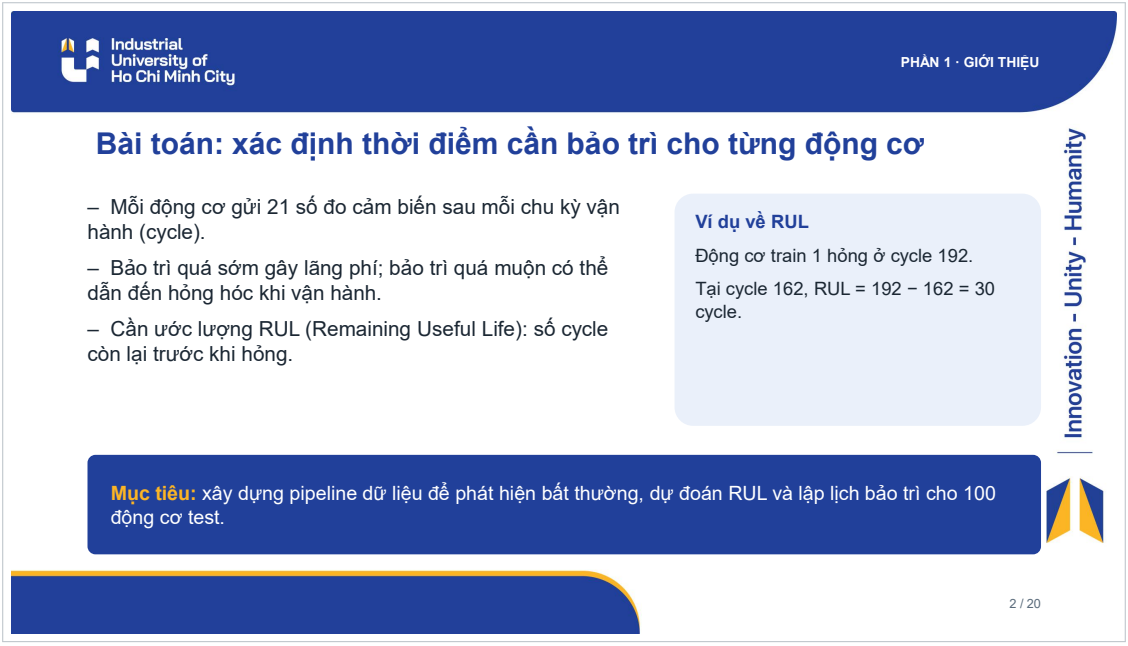
Đọc

Kính chào quý thầy cô. Em là Phan Nguyễn Mai Phương, mã số học viên 25002711. Em trình bày đề tài phân tích dữ liệu IoT và bảo trì dự đoán, dưới sự hướng dẫn của TS. Võ Thị Thanh Vân.

Nói thêm nếu còn thời gian hoặc được hỏi
Đây là Đề tài 8 của môn Phân tích dữ liệu lớn, sử dụng NASA C-MAPSS.

→ *Chuyển slide: Trước hết, em xin trình bày bài toán bảo trì.*

Slide 02/20 Bài toán: xác định thời điểm cần bảo trì cho từng động cơ **35 giây** hết slide lúc 0:55



Nhìn vào

Chỉ các dòng mô tả bài toán, ví dụ RUL và mục tiêu cuối slide.

Đọc

Mỗi động cơ gửi 21 số đo cảm biến sau mỗi cycle vận hành. Bảo trì quá sớm gây lãng phí; quá muộn có thể dẫn đến hỏng khi đang vận hành. Đại lượng cần ước lượng là RUL, tức số cycle còn lại trước khi hỏng. Ví dụ động cơ train 1 hỏng ở cycle 192, nên tại cycle 162, RUL bằng 30. Mục tiêu là lập lịch cho 100 động cơ test. Kết quả xác định **21 động cơ cần bảo trì ngay và cả 21 đều đúng**.

Nói thêm nếu còn thời gian hoặc được hỏi

RUL là Remaining Useful Life. Cycle không quy đổi được thành giờ hay ngày. Kết quả 21/21 không có nghĩa phát hiện đủ mọi động cơ cần bảo trì.

→ Chuyển slide: Để thực hiện mục tiêu này, em xây dựng quy trình gồm năm bước.

Slide 03/20 Quy trình thực hiện gồm năm bước

30 giây hết slide lúc 1:25



Nhìn vào

Chỉ năm bước: Thu thập, Lưu trữ, Xử lý, Phân tích, Kết quả.

Đọc

Quy trình bắt đầu bằng tải và kiểm tra dữ liệu, sau đó lưu theo các lớp có vai trò riêng. Phần xử lý dùng Pandas, PySpark và MapReduce, rồi đối chiếu kết quả trước khi phân tích. Phần cuối gồm phát hiện bất thường, dự đoán RUL và lập lịch bảo trì. Em sẽ trình bày kỹ phần dữ liệu và hạ tầng ở **slide 4–12**, rồi đến phân tích.

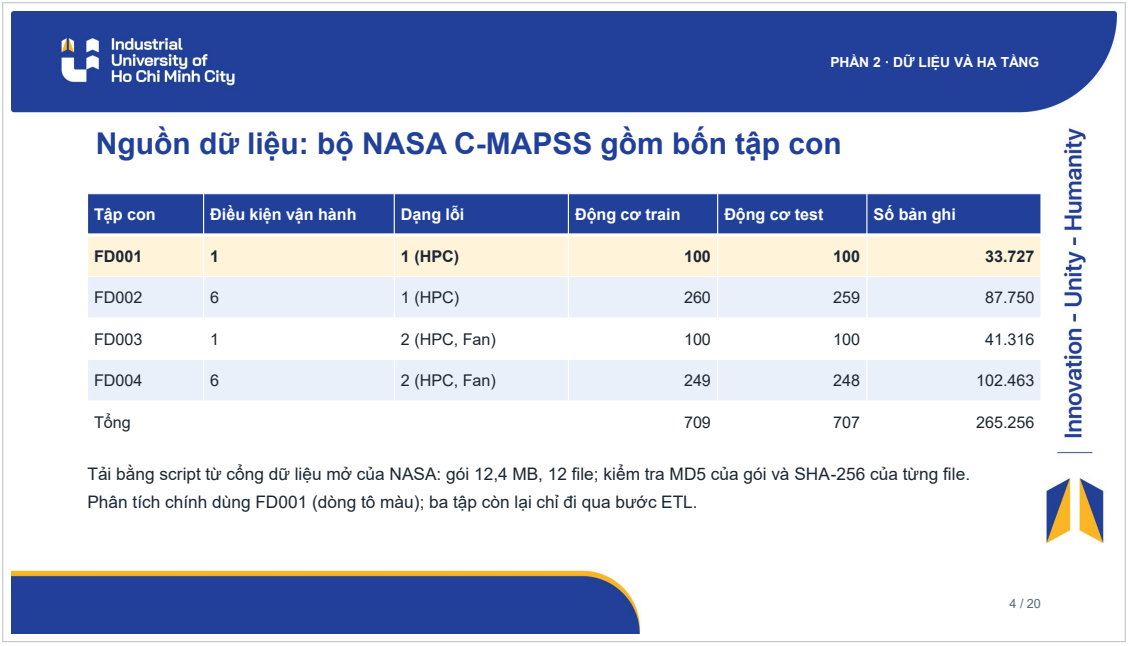
Nói thêm nếu còn thời gian hoặc được hỏi

MD5 và SHA-256 là mã băm kiểm tra tính toàn vẹn và phiên bản dữ liệu. MapReduce đối chiếu bảng tổng hợp, không thay toàn bộ pipeline dự đoán.

→ Chuyển slide: Nguồn dữ liệu của quy trình là bộ NASA C-MAPSS.

Slide 04/20 Nguồn dữ liệu: bộ NASA C-MAPSS gồm bốn tập con

40 giây hết slide lúc 2:05



Nhìn vào

Chỉ thông tin tải dữ liệu và dòng FD001 tô màu trong bảng.

Đọc

C-MAPSS là dữ liệu mô phỏng động cơ turbofan của NASA. Bốn tập con khác nhau về điều kiện vận hành và dạng lỗi. Em dùng FD001 cho phân tích chính; ba tập còn lại chỉ đi qua bước ETL. Dữ liệu được tải bằng script từ NASA, gồm **12 file**, dung lượng gói **12,4 MB**. Em kiểm tra MD5 của gói và SHA-256 của từng file, rồi ghi vào manifest, tức hồ sơ nguồn dữ liệu. Nhờ đó, những lần chạy sau có thể xác nhận đang dùng cùng dữ liệu đầu vào.

Nói thêm nếu còn thời gian hoặc được hỏi

Ngày tải 08/10/2026 theo UTC; tổng cộng 265.256 bản ghi. Nguồn: `https://data.nasa.gov/docs/legacy/CMAPSSData.zip`. Mã băm không thay thế kiểm tra chất lượng từng bản ghi.

→ Chuyển slide: Tiếp theo là cấu trúc bản ghi và vai trò của ba file trong FD001.

Slide 05/20 Mỗi bản ghi là một cycle của một động cơ, gồm 26 trường **40 giây** hết slide lúc 2:45



Nhìn vào

Chỉ phần dữ liệu gốc và ba ô tên file train, test, RUL.

Đọc

File text không có header; các trường ngăn bằng khoảng trắng. Mỗi dòng gồm mã động cơ, cycle, ba thiết lập vận hành và 21 sensor, tổng cộng **26 trường**. Bảng chỉ hiện một phần cột. Train có **20.631 dòng**, quan sát đến khi hỏng, nên tính được nhãn RUL từ cycle cuối. Test có **13.096 dòng**, bị cắt trước khi hỏng. File RUL riêng gồm 100 dòng chỉ dùng đánh giá sau khi đã lưu dự đoán, không dùng làm đầu vào mô hình.

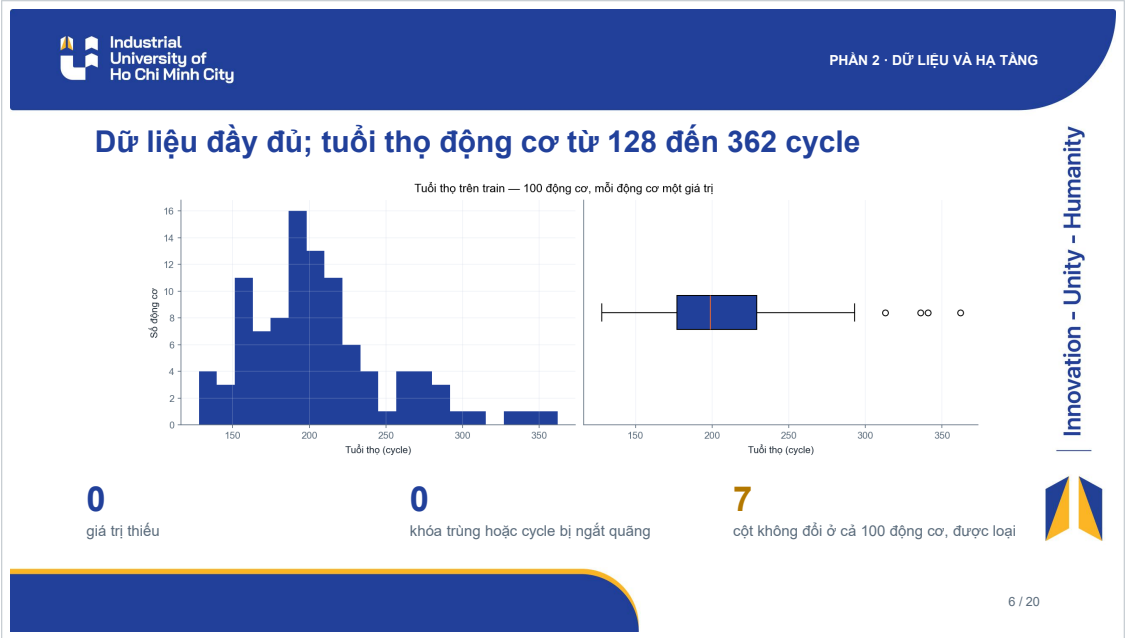
Nói thêm nếu còn thời gian hoặc được hỏi

Một dòng nhãn test là RUL tại cycle quan sát cuối của một động cơ, không phải nhãn cho từng dòng cảm biến trong file test.

→ Chuyển slide: Sau khi đọc file, em kiểm tra chất lượng và tính liên tục của chuỗi.

Slide 06/20 Dữ liệu đầy đủ; tuổi thọ động cơ từ 128 đến 362 cycle

40 giây hết slide lúc 3:25



Nhìn vào

Chỉ các số 0, 0, 7 và biểu đồ tuổi thọ.

Đọc

FD001 không có giá trị thiếu, khóa trùng hoặc cycle ngắt quãng. Khóa bảng quan sát là **split, engine_id, cycle**: cần split vì cùng mã động cơ và cycle có thể xuất hiện ở cả train lẫn test. Các cột không đổi không cung cấp thông tin suy giảm. Tuổi thọ train trải từ **128 đến 362 cycle**. Nếu dùng chung mốc trung vị 199 cycle để bảo trì, 49 động cơ đã hỏng trước đó. Vì vậy cần dự đoán riêng cho từng động cơ.

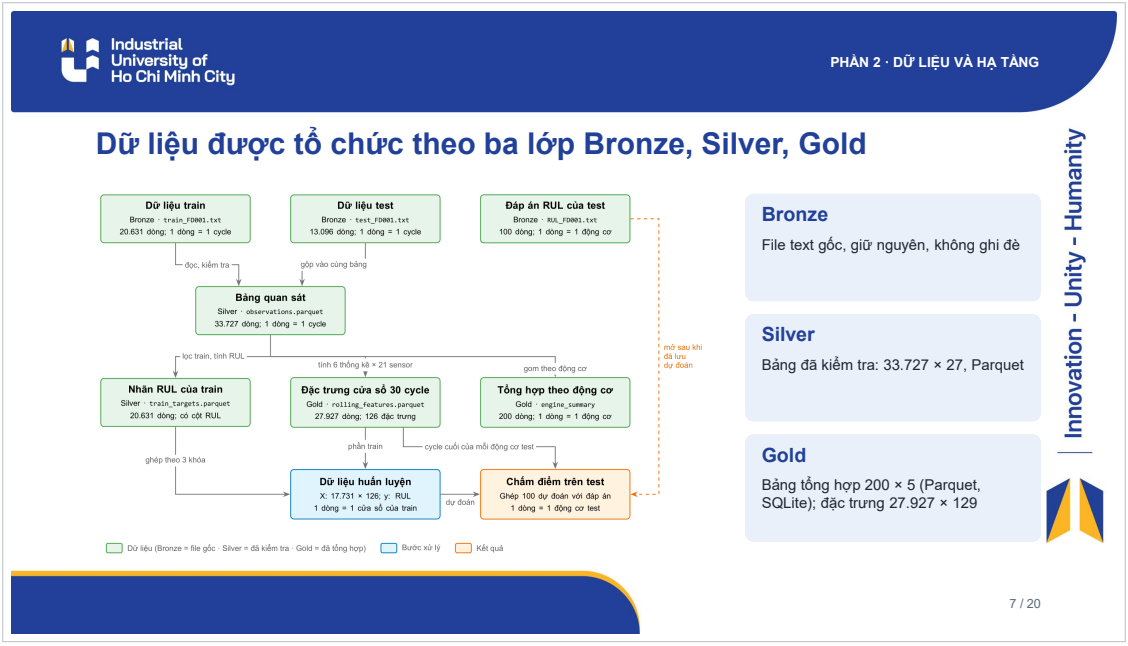
Nói thêm nếu còn thời gian hoặc được hỏi

Bảy cột không đổi gồm sensor 1, 5, 10, 16, 18, 19 và setting_3. Sensor 6 không đổi ở 38/100 động cơ. Không hiểu việc loại cột là xóa khỏi dữ liệu raw: làm mượt dùng sensor có thông tin; bảng đặc trưng RUL vẫn giữ 21 sensor.

→ Chuyển slide: Dữ liệu đã kiểm tra được tổ chức thành ba lớp.

Slide 07/20 Dữ liệu được tổ chức theo ba lớp Bronze, Silver, Gold

45 giây hết slide lúc 4:10



Nhìn vào

Chỉ ba lớp và kích thước Silver, bảng tổng hợp, bảng đặc trưng.

Đọc

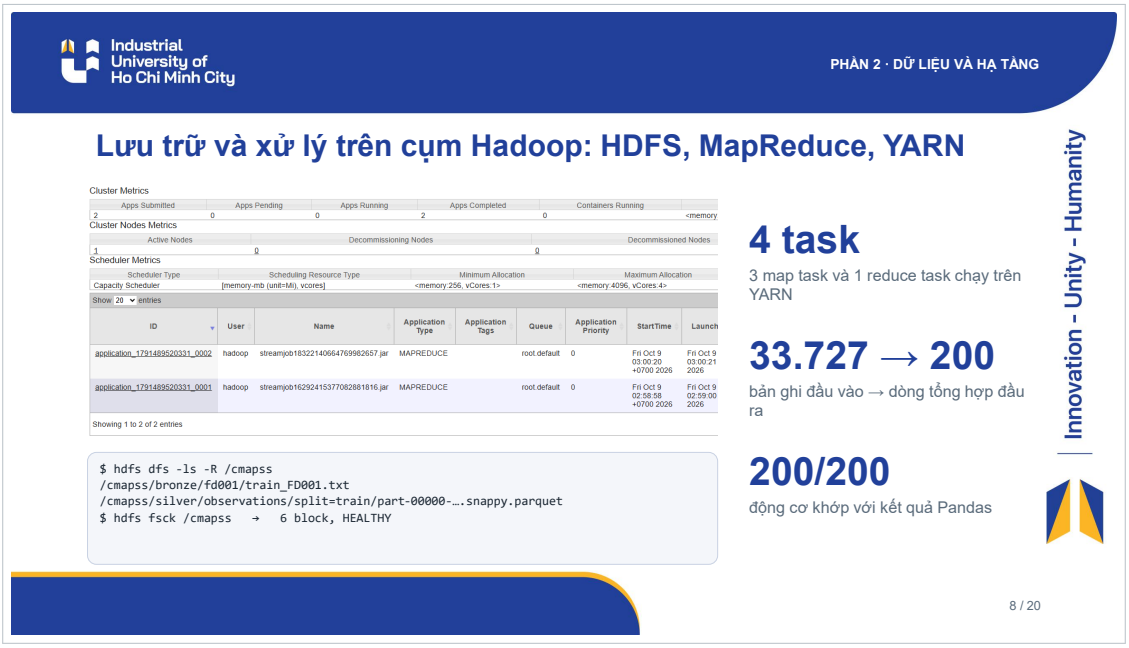
Bronze giữ file gốc, không ghi đè. Silver gộp train và test theo dòng, bổ sung split và kiểm tra kiểu, khóa; kết quả là **33.727 dòng, 27 cột**. Gold chứa bảng tổng hợp 200 động cơ và bảng đặc trưng 27.927 dòng. Em dùng Parquet vì lưu theo cột, phù hợp khi chỉ cần đọc một số sensor. Bảng nhỏ còn được lưu bằng SQLite để truy vấn SQL. Mỗi lần ghi qua thư mục tạm, kiểm tra xong mới thay bộ cũ, đồng thời lưu mã băm đầu vào để truy vết kết quả.

Nói thêm nếu còn thời gian hoặc được hỏi

Bảng tổng hợp có 200 dòng × 5 cột; bảng đặc trưng có 27.927 dòng × 129 cột. Khóa bảng tổng hợp là (split, engine_id); bảng quan sát và đặc trưng có thêm cycle.

→ Chuyển slide: Ngoài lưu trên máy, em triển khai xử lý trên Hadoop.

Slide 08/20 Lưu trữ và xử lý trên cụm Hadoop: HDFS, MapReduce, YARN **50 giây** hết slide lúc 5:00



Nhìn vào

Chỉ đường dẫn HDFS, HEALTHY, rồi 4 task, 33.727 → 200 và 200/200.

Đọc

Hadoop chạy trong Docker trên một node. HDFS lưu trữ dữ liệu; YARN quản lý tài nguyên chạy tác vụ; MapReduce thực hiện tổng hợp. Hai file raw được đưa lên HDFS. Pha map đọc bản ghi theo động cơ; pha reduce tổng hợp số cycle và giới hạn cycle của từng động cơ. Job chạy **3 map task và 1 reduce task**, chuyển 33.727 bản ghi thành 200 dòng. Spark cũng đọc HDFS và ghi Silver dạng Parquet. Cả hai cách cho bảng khớp **200/200 động cơ** với Pandas. Một node xác nhận luồng xử lý hoạt động, chưa chứng minh lợi ích phân tán nhiều máy.

Nói thêm nếu còn thời gian hoặc được hỏi

Hadoop 3.4.3 gồm NameNode, DataNode, ResourceManager và NodeManager. MapReduce dùng Hadoop Streaming, mất 22,4 giây; bước Spark mất 14,1 giây. Lệnh fsck ghi nhận 6 block, trạng thái HEALTHY.

→ Chuyển slide: Với phần tạo đặc trưng, em tiếp tục đối chiếu PySpark và Pandas.

Slide 09/20 PySpark cho kết quả trùng khớp với Pandas

45 giây hết slide lúc 5:45



Nhìn vào

Chỉ partitionBy, orderBy và rowsBetween trong đoạn code.

Đọc

Em không chỉ so số dòng. Bộ đối chiếu kiểm tra schema, tức cấu trúc cột, rồi khóa và từng giá trị. Ba bảng ETL khớp tuyệt đối; bảng đặc trưng **27.927 dòng** khớp trong sai số cho phép. Đoạn code chia theo split và động cơ, sắp theo cycle, rồi lấy 30 dòng gần nhất. Vì cycle đã được kiểm tra liên tục, đó là **window 30 cycle**. Các phép trung bình, độ lệch chuẩn và độ dốc tương đương bản Pandas. Đây là căn cứ thay công cụ xử lý mà giữ cách tính.

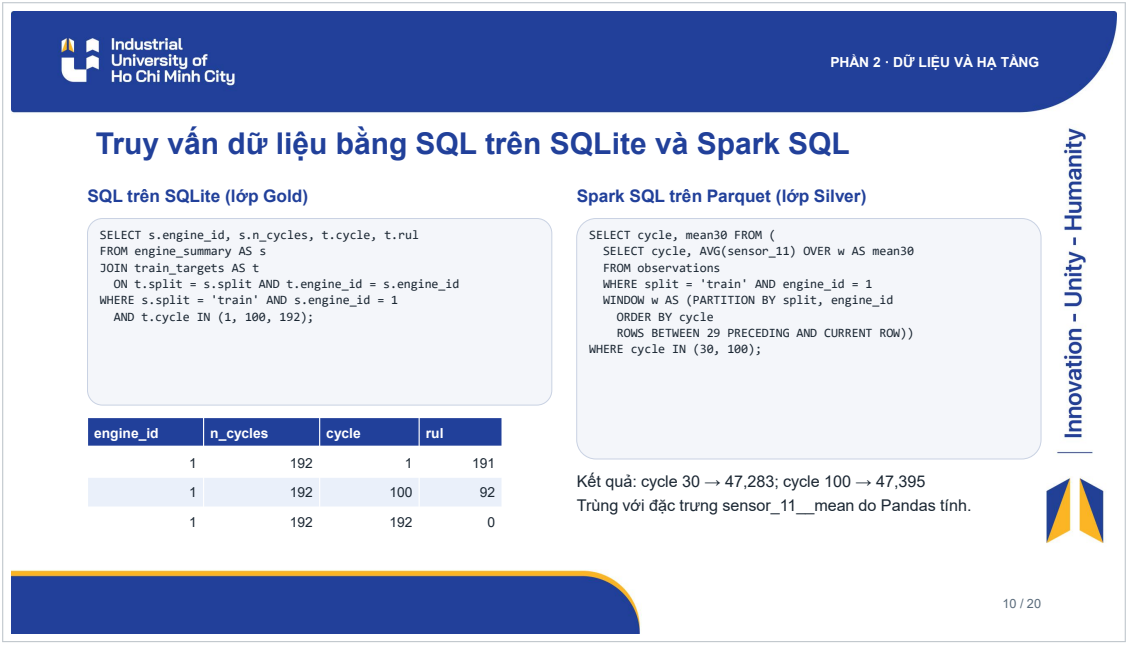
Nói thêm nếu còn thời gian hoặc được hỏi

Sai số cho phép của đặc trưng là 10^{-7}. `unionByName` gộp theo tên cột; `stddev_samp` tính độ lệch chuẩn mẫu. Parity, tức đối chiếu kết quả, không chứng minh tốc độ hoặc khả năng chịu lỗi.

→ Chuyển slide: Các bảng còn có thể kiểm tra trực tiếp bằng SQL.

Slide 10/20 Truy vấn dữ liệu bằng SQL trên SQLite và Spark SQL

45 giây hết slide lúc 6:30



Nhìn vào

Chỉ JOIN bên trái và vị trí WHERE ngoài window bên phải.

Đọc

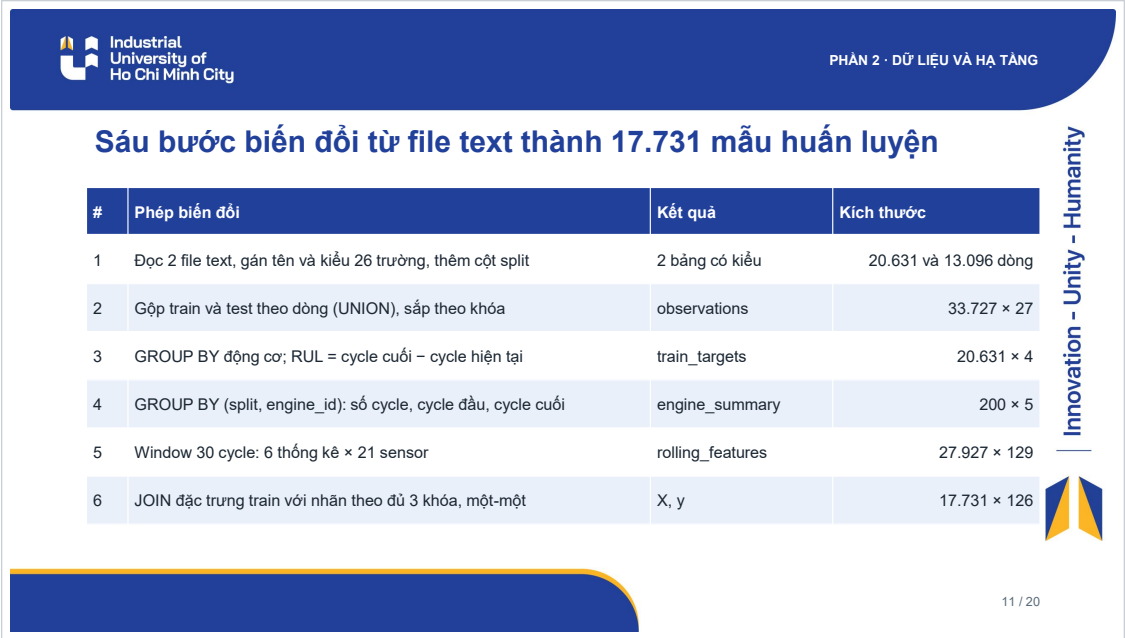
Bên trái, SQLite ghép bảng tổng hợp với nhãn theo split và mã động cơ, lấy RUL của động cơ train 1 tại các cycle đã chọn. Bên phải, Spark SQL đọc Parquet và tính trung bình sensor 11 trong **30 cycle gần nhất**; kết quả trùng Pandas. Điểm quan trọng là lọc cycle ở truy vấn ngoài, sau khi tính window. Nếu lọc trước, phép tính chỉ nhìn thấy những dòng còn lại và trung bình sẽ sai. Ví dụ tại cycle 100, phải lấy cả đoạn lịch sử, không chỉ dòng cycle 100.

Nói thêm nếu còn thời gian hoặc được hỏi

Trung bình là 47,283 tại cycle 30 và 47,395 tại cycle 100. Có thể lọc split và engine_id trước vì vẫn giữ nguyên chuỗi đang xét; điều kiện chọn cycle trình bày phải đặt sau phép tính window.

→ Chuyển slide: Từ những phép xử lý đó, em tạo bảng mẫu huấn luyện.

Slide 11/20 Sáu bước biến đổi từ file text thành 17.731 mẫu huấn luyện **45 giây** hết slide lúc 7:15



Nhìn vào

Chỉ tiêu đề 17.731 mẫu và các bước biến đổi trong bảng.

Đọc

Đây là phần biến đổi trong ETL, tức quy trình đọc, biến đổi và ghi dữ liệu. Mỗi mẫu gồm sáu thống kê của 21 sensor trên 30 cycle gần nhất: giá trị cuối, trung bình, độ lệch chuẩn, nhỏ nhất, lớn nhất và độ dốc. Như vậy có **126 đặc trưng**. Mỗi động cơ mất 29 cycle đầu vì chưa đủ window, nên từ 20.631 dòng train còn **17.731 mẫu**. Ghép nhãn phải dùng đủ split, engine_id và cycle với quan hệ một-một, để một mẫu không bị ghép với nhiều nhãn.

Nói thêm nếu còn thời gian hoặc được hỏi

Nếu chỉ ghép theo engine_id, động cơ train 1 có thể thành 163 × 192 = 31.296 dòng thay vì 163 dòng. Bước 1–5 dùng cả Pandas và PySpark; kiểm tra quan hệ ghép ở bước 6 ngăn nhân dòng ngoài ý muốn.

→ Chuyển slide: Sau tính đúng, em kiểm tra thời gian xử lý khi tăng quy mô.

Slide 12/20 Khi dữ liệu tăng 100 lần, Spark tính nhanh hơn Pandas 5,5 lần **50 giây** hết slide lúc 8:05



Nhìn vào

Chỉ 5,5 lần, dấu x ở mức 200× và kết luận lịch bảo trì như nhau.

Đọc

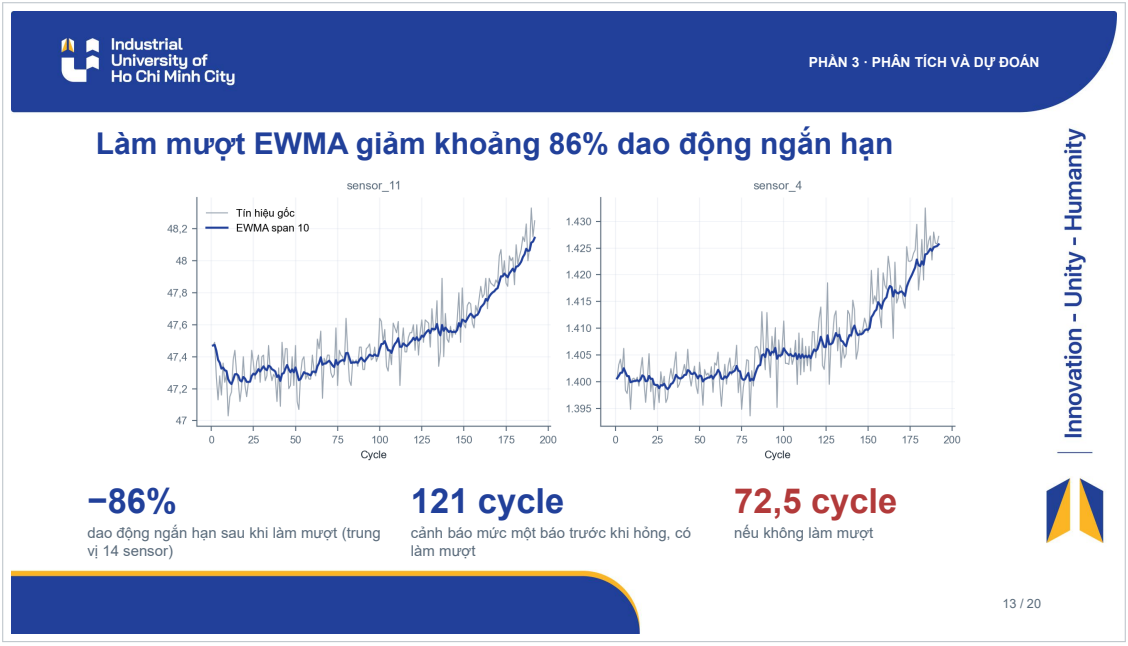
Em nhân bản FD001 với mã động cơ mới rồi cho hai công cụ cùng đọc, tổng hợp và tạo đặc trưng. Thời gian chỉ gồm **tính toán, không gồm ghi file**. Ở mức 100 lần, Spark nhanh hơn Pandas **5,5 lần**. Ở mức 200 lần, Pandas chưa xong sau 30 phút, còn Spark xong sau 392 giây. Đây là phép đo trên một máy: Pandas một luồng, Spark bốn luồng, nên một phần chênh lệch đến từ song song hóa. Với dữ liệu hiện tại, Pandas đủ dùng và lịch bảo trì không đổi. Spark có lợi ở các quy mô lớn hơn đã đo.

Nói thêm nếu còn thời gian hoặc được hỏi

Các mức đo: 1, 10, 50, 100, 200 lần. Tại 100×: Pandas 1.075 giây, Spark 196 giây. Bước ghi đầy đủ của Spark thất bại từ 50× do phải gom về driver trong cách triển khai hiện tại; không được hiểu là pipeline đầy đủ thành công ở 200×. Giới hạn 30 phút do thực nghiệm đặt ra.

→ *Chuyển slide: Em chuyển sang phân tích, bắt đầu bằng làm mượt tín hiệu.*

Slide 13/20 Làm mượt EWMA giảm khoảng 86% dao động ngắn hạn **35 giây** hết slide lúc 8:40



Nhìn vào

Chỉ tín hiệu gốc, sau làm mượt và ba số 86%, 121, 72,5 cycle.

Đọc

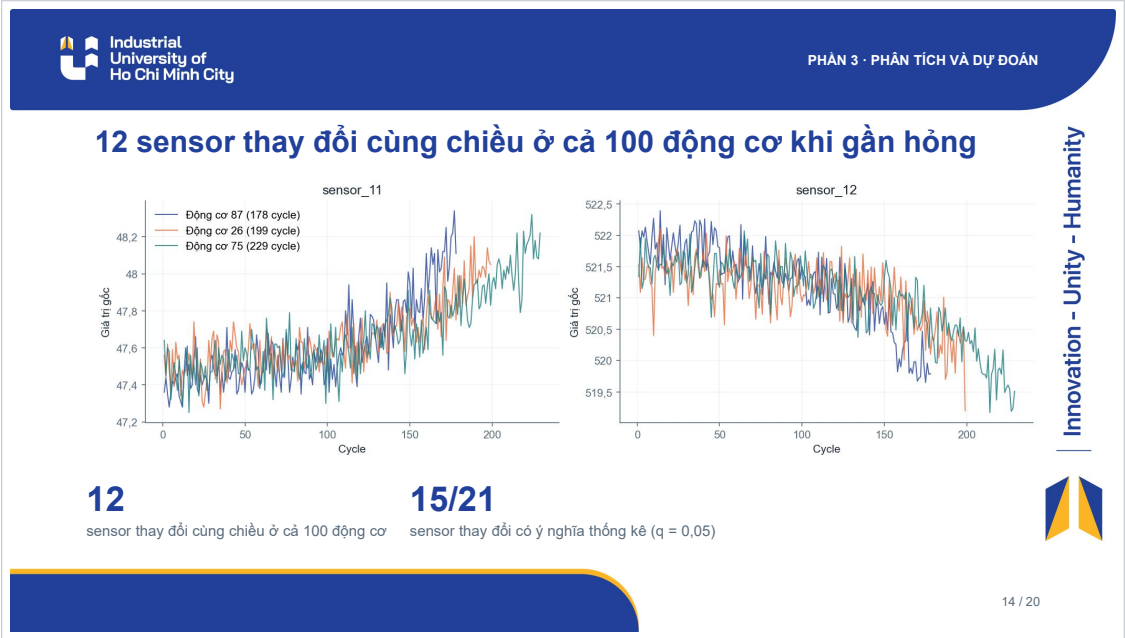
EWMA là trung bình có trọng số giảm dần theo thời gian: quan sát gần hiện tại có trọng số lớn hơn, chỉ dùng hiện tại và quá khứ. Trên train, dao động ngắn hạn giảm khoảng **86%**, tính theo trung vị 14 sensor. Đây không phải mức tăng độ chính xác. Sau làm mượt, cảnh báo mức một có thời gian báo trước trung vị **121 cycle**, so với 72,5 cycle nếu không làm mượt.

Nói thêm nếu còn thời gian hoặc được hỏi

Span 10 ứng với $\alpha = 2/(10+1) \approx 0{,}18$. Độ lệch chuẩn sai phân sensor 4 giảm từ 5,70 xuống 0,78. Mức giảm trên 14 sensor nằm trong 83,6–86,4%.

→ Chuyển slide: Tiếp theo là kiểm tra thay đổi của sensor khi gần hỏng.

Slide 14/20 12 sensor thay đổi cùng chiều ở cả 100 động cơ khi gần hỏng **30 giây** hết slide lúc 9:10



Nhìn vào

Chỉ đường sensor 11, sensor 12 và hai kết quả 12, 15/21.

Đọc

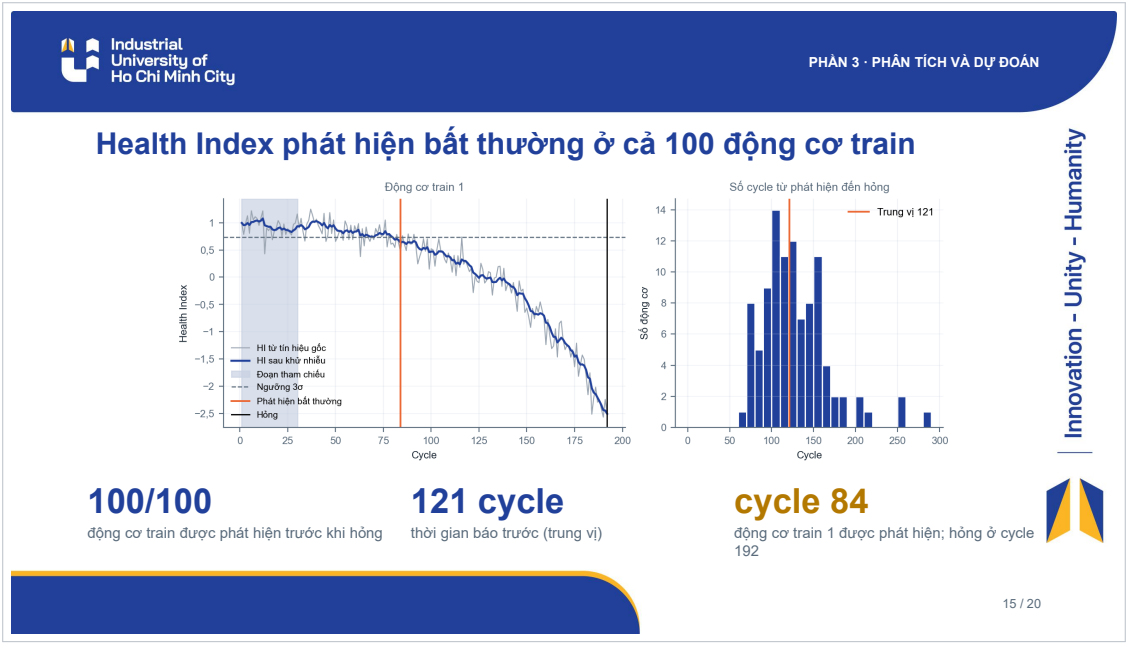
Em so trung bình 30 cycle cuối với 30 cycle đầu của từng động cơ. Có **12 sensor** thay đổi cùng chiều ở cả 100 động cơ train: sensor 11 tăng, sensor 12 giảm. Sau kiểm định và hiệu chỉnh nhiều phép kiểm định, **15/21 sensor** thay đổi có ý nghĩa thống kê. Đây là phân tích mô tả dấu hiệu suy giảm, không dùng nhãn test để chọn sensor cho mô hình.

Nói thêm nếu còn thời gian hoặc được hỏi

Dùng Wilcoxon, bootstrap 10.000 lần và hiệu chỉnh Benjamini–Hochberg với $q = 0{,}05$. Sáu sensor không đổi nên không thực hiện được kiểm định chênh lệch.

→ Chuyển slide: Từ nhiều sensor, em xây dựng chỉ số tổng hợp phát hiện bất thường.

Slide 15/20 Health Index phát hiện bất thường ở cả 100 động cơ train **45 giây** hết slide lúc 9:55



Nhìn vào

Chỉ cycle 84 của động cơ train 1 và các số 100/100, 121 cycle.

Đọc

Health Index, viết tắt HI, là chỉ số sức khỏe tổng hợp từ sensor. Em chuẩn hóa rồi dùng PCA, tức phân tích thành phần chính, lấy một thành phần biểu diễn trạng thái. Bước này không dùng nhãn RUL. Mức tham chiếu là 30 cycle đầu của từng động cơ. Khi HI giảm quá ba độ lệch chuẩn trong năm cycle liên tiếp, hệ thống phát **cảnh báo mức một**. Động cơ train 1 được phát hiện ở cycle 84, hỏng ở cycle 192. Cả 100 động cơ train được phát hiện, với thời gian báo trước trung vị 121 cycle.

Nói thêm nếu còn thời gian hoặc được hỏi

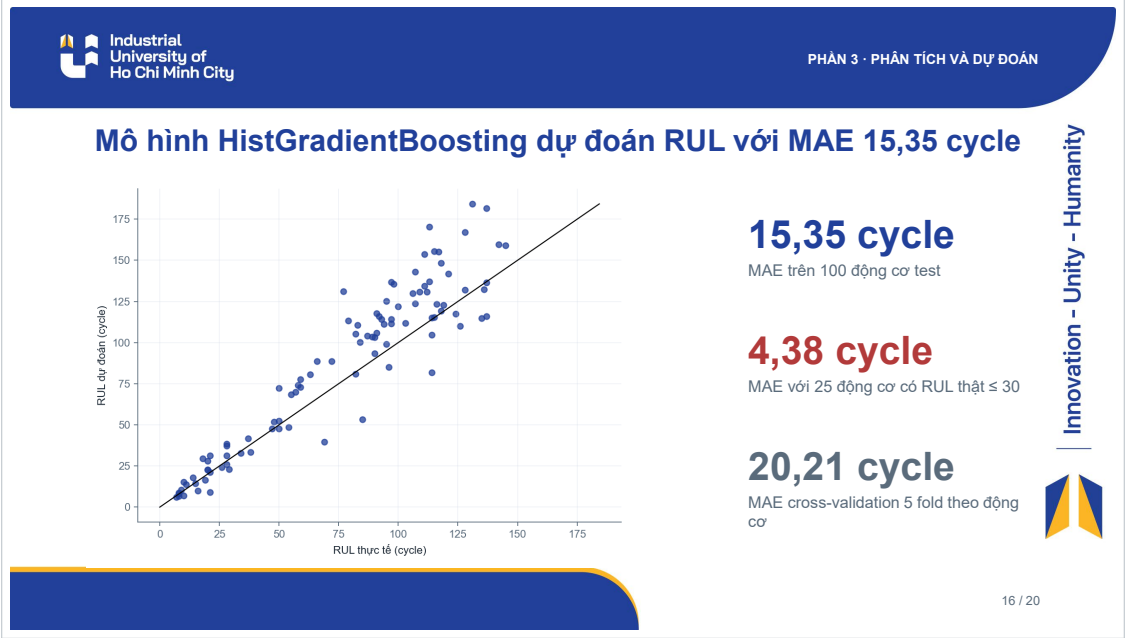
Scaler và PCA chỉ học trên train. Thời gian báo trước nhỏ nhất 67, lớn nhất 289 cycle. Lưới chín cấu hình đều phát hiện đủ 100 động cơ. Chưa đo được báo động giả do thiếu nhãn bắt đầu suy giảm và chuỗi bình thường đối chứng.

→ *Chuyển slide: Thời điểm bảo trì còn cần mô hình ước lượng RUL.*

Slide 16/20 Mô hình HistGradientBoosting dự đoán RUL với MAE 15,35 cycle

45 giây hết

slide lúc 10:40



Nhìn vào

Chỉ đường chéo và ba số MAE test, vùng gần hỏng, cross-validation.

Đọc

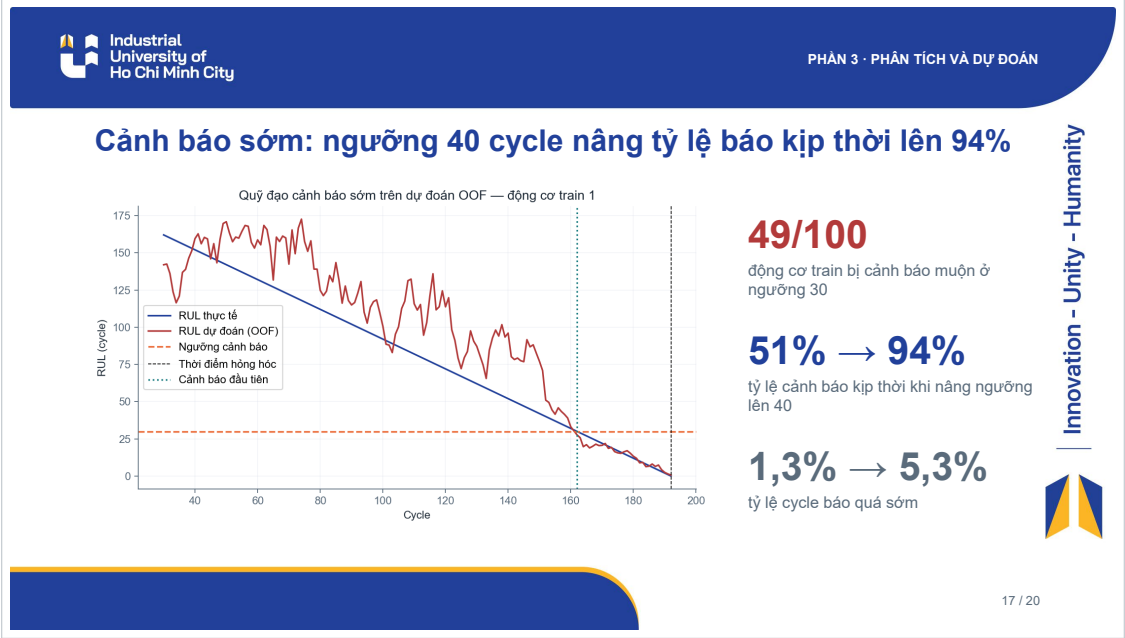
HistGradientBoosting, gọi tắt HGB, được chọn bằng cross-validation năm fold theo động cơ: mọi window của một động cơ nằm cùng một fold. MAE, tức sai số tuyệt đối trung bình, trong cross-validation là 20,21 cycle. Trên test, mỗi điểm là một động cơ được dự đoán tại cycle cuối; MAE là **15,35 cycle**. Với 25 động cơ có RUL thật không quá 30, MAE là **4,38 cycle**. Hai cách đánh giá khác thời điểm lấy mẫu nên không so hai MAE như cùng một tập.

Nói thêm nếu còn thời gian hoặc được hỏi

HGB thấp hơn Ridge 3,78 cycle trong cross-validation; baseline trung vị là 46,95. Test có RMSE 20,54 cycle; khoảng tin cậy 95% của MAE là [12,73; 18,09]. Mô hình chính không dùng RUL cap.

→ Chuyển slide: Sai số RUL ảnh hưởng trực tiếp đến thời điểm phát cảnh báo.

Slide 17/20 Cảnh báo sớm: ngưỡng 40 cycle nâng tỷ lệ báo kịp thời lên 94% **45 giây** hết slide lúc 11:25



Nhìn vào

Chỉ quỹ đạo động cơ train 1 và 49/100, 51% → 94%, 1,3% → 5,3%.

Đọc

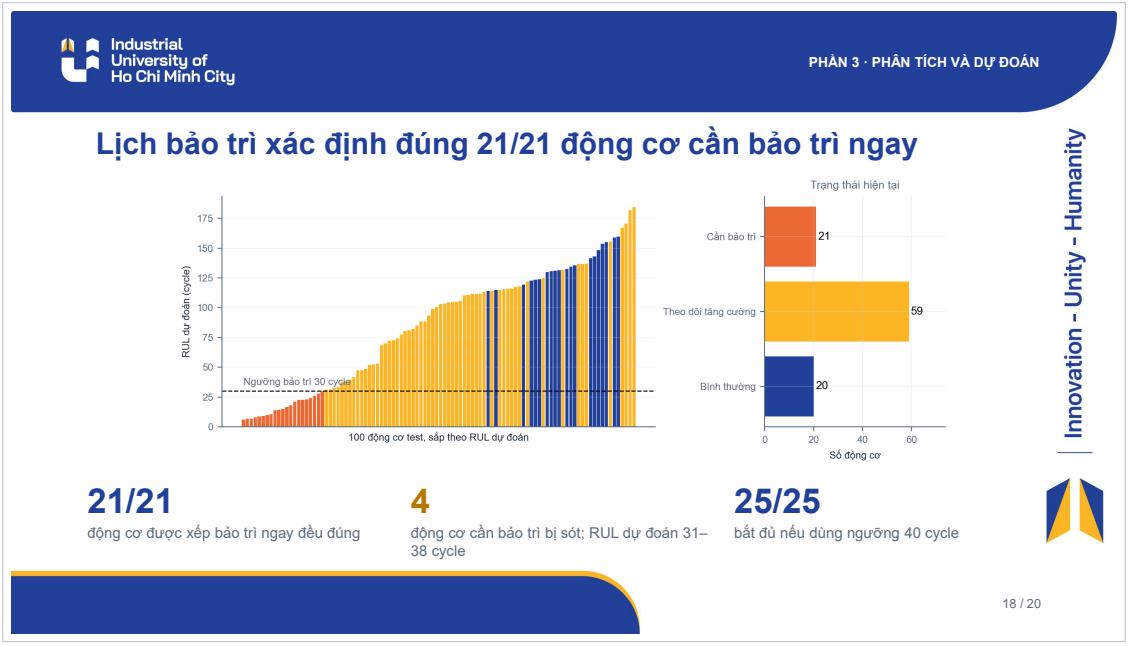
Cảnh báo mức hai phát khi RUL dự đoán không quá 30 cycle. Để đánh giá thời điểm báo, em dùng dự đoán OOF trên train: mỗi động cơ được dự đoán bởi mô hình không học từ động cơ đó. Ở ngưỡng 30, **49/100 động cơ** bị báo muộn so với mốc cần báo trước, dù vẫn trước khi hỏng. Nâng ngưỡng lên 40 làm tỷ lệ kịp thời tăng từ **51% lên 94%**, nhưng tỷ lệ cycle báo quá sớm cũng tăng. Đánh đổi này cần hiệu chuẩn theo chi phí bảo trì.

Nói thêm nếu còn thời gian hoặc được hỏi

Ngưỡng 30 có 44 báo trước, 7 đúng lúc, 49 muộn. Tỷ lệ cycle báo quá sớm tăng từ 1,29% lên 5,33%. Trên test, 74/100 dự đoán cao hơn RUL thật; đây là xu hướng dự đoán dư, không phải tỷ lệ cảnh báo muộn trên test.

→ Chuyển slide: Hai mức cảnh báo được kết hợp thành lịch bảo trì cho test.

Slide 18/20 Lịch bảo trì xác định đúng 21/21 động cơ cần bảo trì ngay **50 giây** hết slide lúc 12:15



Nhìn vào

Chỉ 21/21, 4 và 25/25; phân biệt số báo đúng với tổng số cần bảo trì.

Đọc

Trên 100 động cơ test, hệ thống xếp **21 cần bảo trì ngay**, 59 theo dõi tăng cường và 20 bình thường. Đối chiếu nhãn sau khi lưu lịch, cả 21 chỉ định bảo trì ngay đều đúng. Tuy nhiên, có 25 động cơ thực sự cần bảo trì, nên còn **4 trường hợp** chưa được xếp bảo trì ngay. Ví dụ động cơ 90 ở cycle 146, RUL dự đoán 31,2, lịch tại cycle 147; cảnh báo mức một đã xuất hiện từ cycle 67. Nếu dùng ngưỡng 40 thì phát hiện đủ 25, đổi lại ba lần báo sớm.

Nói thêm nếu còn thời gian hoặc được hỏi

Bốn trường hợp có RUL dự đoán 31,22–38,46 cycle. Lịch chính giữ ngưỡng 30 chốt trước khi xem test. Có 12 động cơ có lịch dự kiến muộn hơn lúc hỏng 4–27 cycle; chưa kiểm chứng cập nhật lịch sẽ khắc phục kịp các trường hợp đó.

→ *Chuyển slide: Em xin minh họa cách đọc kết quả trên dashboard trong khoảng hai phút.*

Demo dashboard (2 phút, sau slide 18)

Thứ tự 5 bước: Lịch bảo trì → Khử nhiễu → Xu hướng & bất thường → Cảnh báo sớm → Tổng quan. Các trang chọn ở thanh bên trái, mục "Điều hướng". Không mở terminal và không chạy lại huấn luyện trước lớp.

Bước 1 – Lịch bảo trì (45 giây, hết lúc 13:00)



Bấm

Thanh trái: bấm **Lịch bảo trì**. Chỉ vào bốn ô số trên cùng, rồi bảng.

Đọc

Đây là trang lịch bảo trì. Các ô phía trên cho biết **21** động cơ cần bảo trì ngay, **59** theo dõi tăng cường, **20** bình thường và **14** đến hạn trong 25 cycle tới. Bảng bên dưới được sắp theo mức ưu tiên bảo trì.



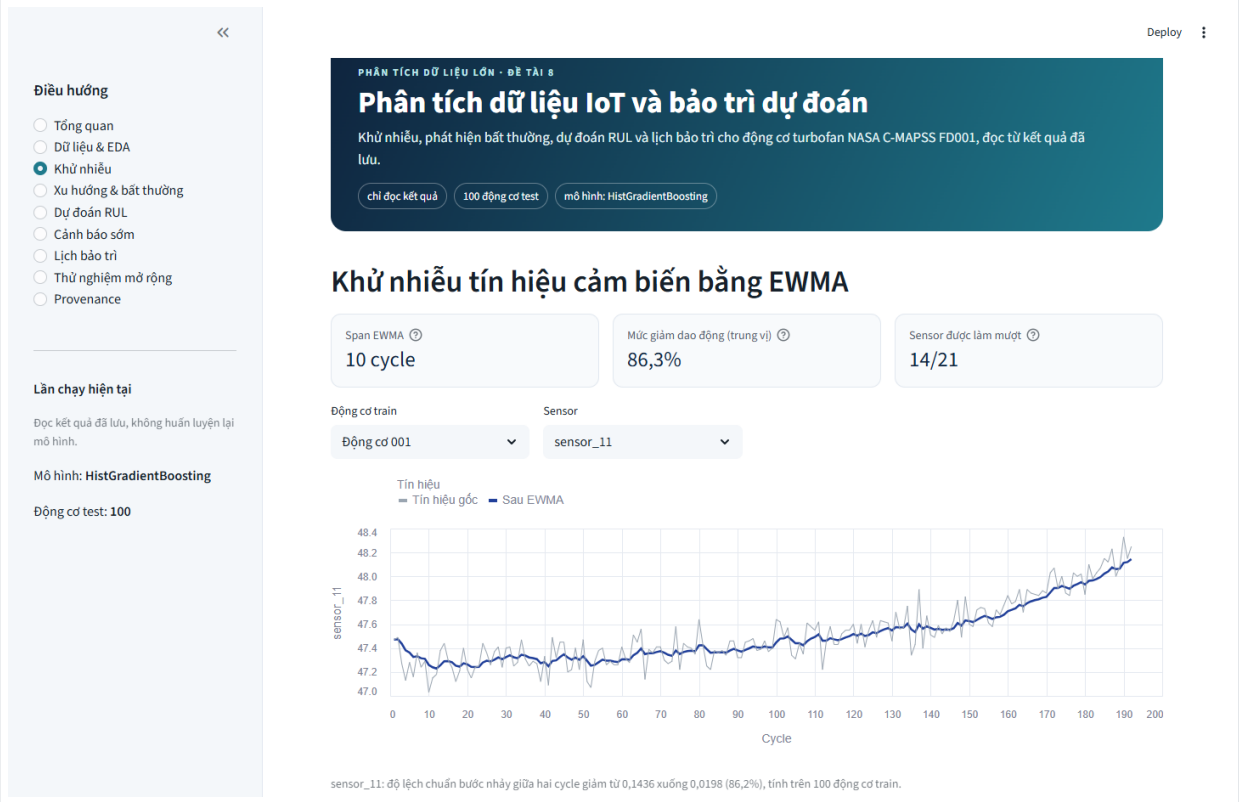
Bấm

Kéo xuống. Ô **"Xem chi tiết động cơ test"** bên phải: bấm vào, gõ `090`, nhấn Enter. Chỉ ô 31,2 và 147, rồi vạch cam trên biểu đồ. Kéo xuống bảng "Kiểm tra bằng nhãn test".

Đọc

Động cơ test 90 có RUL dự đoán **31,2**, RUL thật 28, lịch ở cycle **147**, cảnh báo mức một từ cycle **67**. Bảng cuối trang đối chiếu với nhãn test: **21 chỉ định bảo trì ngay đều đúng**.

Bước 2 – Khử nhiễu (20 giây, hết lúc 13:20)



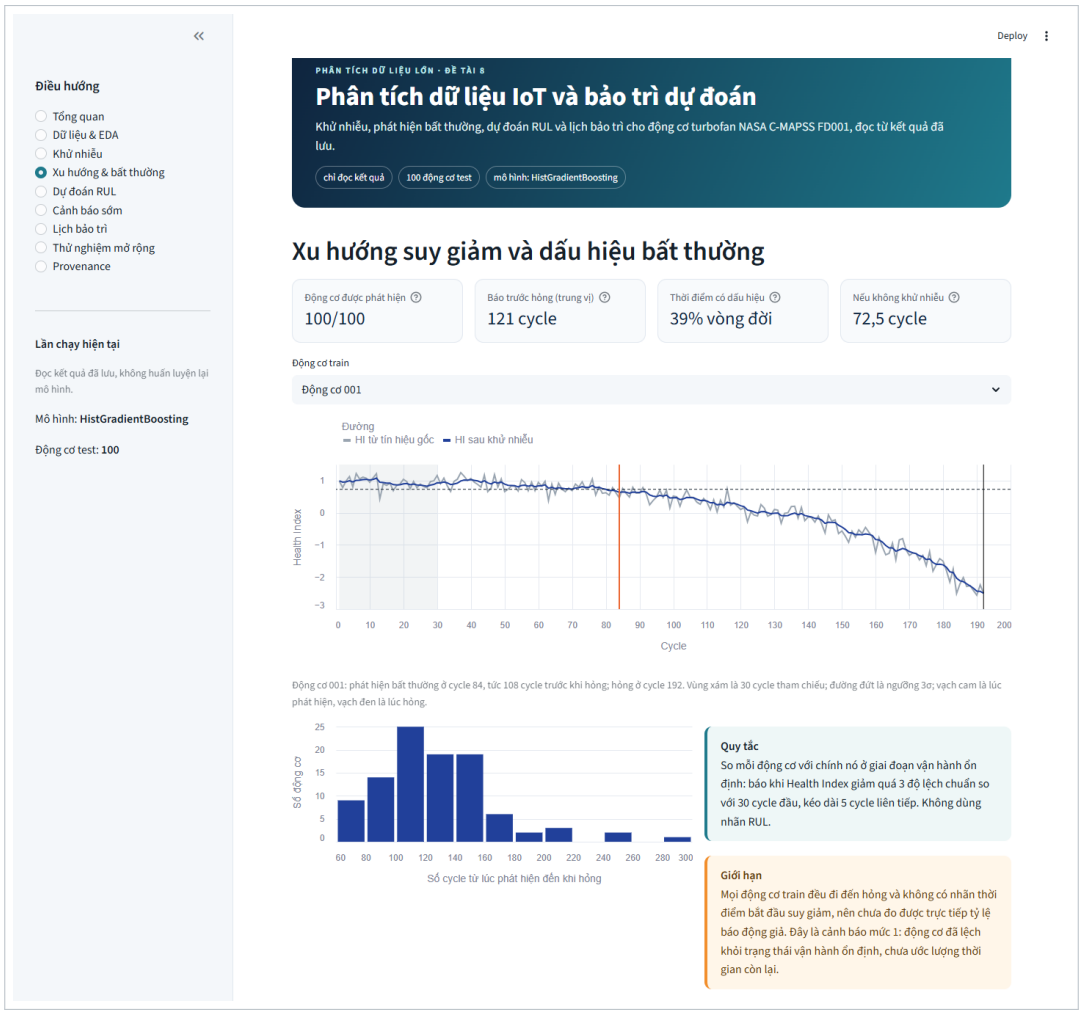
Bấm

Thanh trái: bấm **Khử nhiễu**. Giữ Động cơ 001 và sensor_11. Chỉ đường xám rồi đường xanh. Nếu được hỏi thêm, đổi ô Sensor sang `sensor_4`.

Đọc

Đường xám là tín hiệu gốc, đường xanh là tín hiệu sau EWMA. Với động cơ train 1 và sensor 11 đang chọn, xu hướng rõ hơn sau khi giảm dao động ngắn hạn.

Bước 3 – Xu hướng và bất thường (25 giây, hết lúc 13:45)



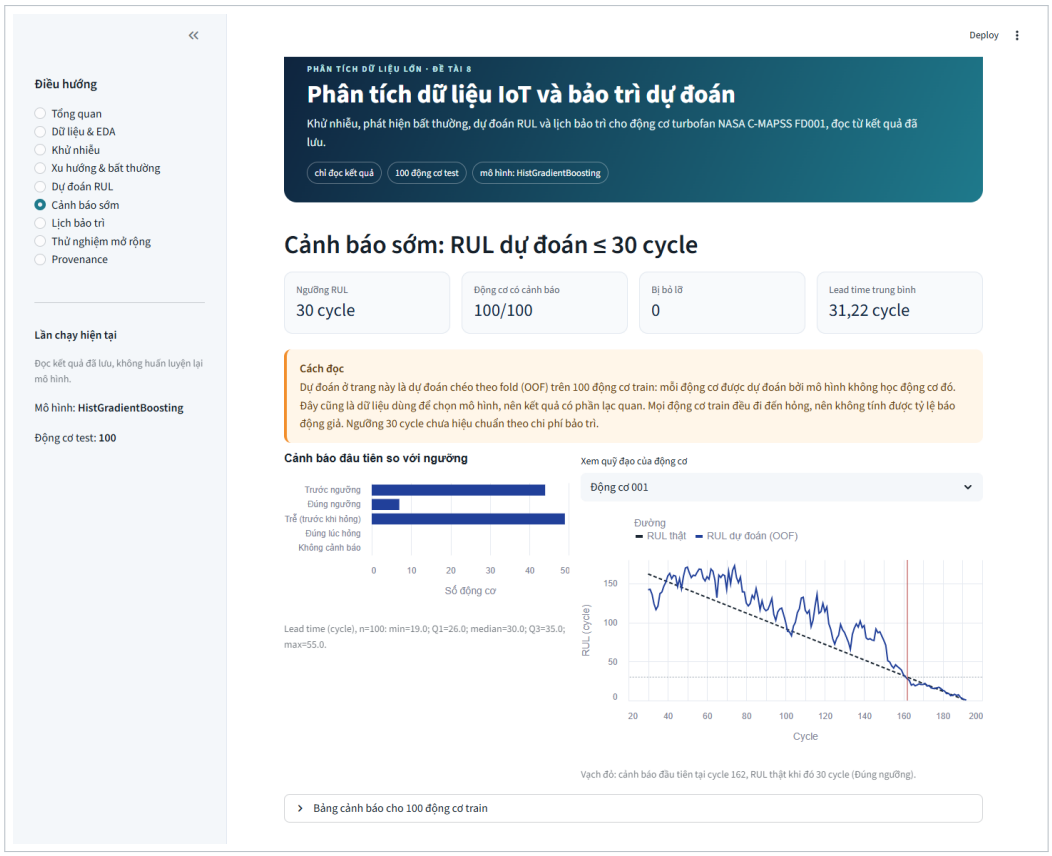
Bấm

Thanh trái: bấm **Xu hướng & bất thường**. Giữ Động cơ 001. Chỉ vùng xám → đường đứt → vạch cam → vạch đen, rồi histogram bên dưới.

Đọc

Cảnh báo mức một của động cơ train 1 xuất hiện ở cycle **84**, còn **108** cycle nữa mới hỏng. Biểu đồ dưới cho thấy phân phối thời gian báo trước của 100 động cơ train; không phải mọi động cơ đều có cùng thời gian báo trước.

Bước 4 – Cảnh báo sớm (20 giây, hết lúc 14:05)



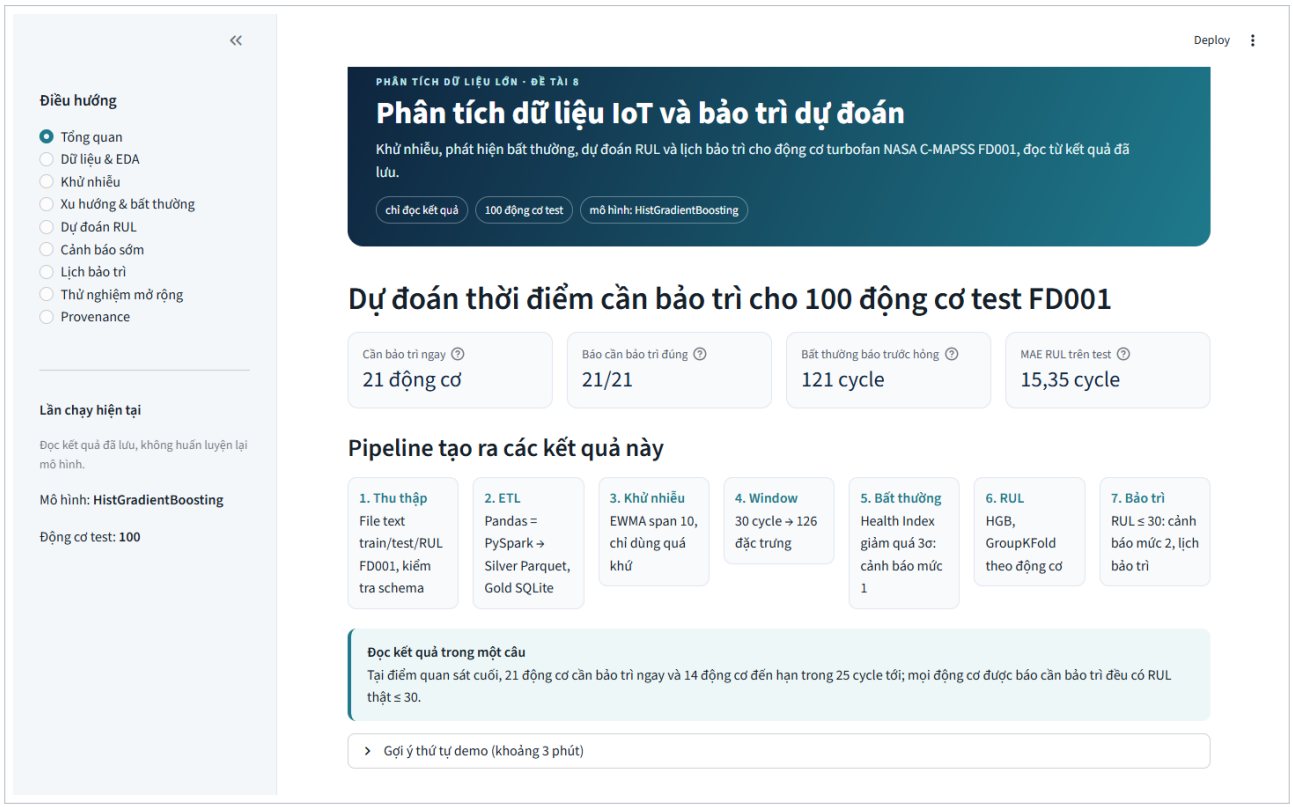
Bấm

Thanh trái: bấm **Cảnh báo sớm**. Ô "Xem quỹ đạo của động cơ": chọn `Động cơ 001`. Chỉ vạch đứng ở cycle 162.

Đọc

Cùng động cơ train 1, cảnh báo mức hai xuất hiện ở cycle **162**. Cảnh báo mức một ở 84, mức hai ở 162, và hỏng ở 192.

Bước 5 – Kết demo (10 giây, hết lúc 14:15)



Bấm

Thanh trái: bấm **Tổng quan**. Sau đó `Alt+Tab` về PowerPoint, sang slide 19.

Đọc

Mọi con số trên dashboard đều đọc từ cùng kết quả với báo cáo và slide, không nhập tay số liệu. Em xin quay lại phần kết luận.

Slide 19/20 Kết luận, hạn chế và hướng phát triển

35 giây hết slide lúc 14:50



Nhìn vào

Chỉ ba nhóm Kết luận, Hạn chế và Hướng phát triển.

Đọc

Báo cáo xây dựng được quy trình dữ liệu có kiểm tra và đối chiếu giữa các công cụ. Health Index hỗ trợ phát hiện bất thường; mô hình RUL và lịch bảo trì cho kết quả khả quan ở vùng gần hỏng. Giới hạn là dữ liệu mô phỏng, chưa có chi phí bảo trì, mô hình còn dự đoán dư RUL và hạ tầng mới chạy trên một máy. Hướng tiếp theo là chọn ngưỡng theo chi phí, kiểm tra trên cụm nhiều node và tiếp nhận dữ liệu streaming, tức dữ liệu đến liên tục.

Nói thêm nếu còn thời gian hoặc được hỏi

Parity chỉ áp dụng cho các bảng đã kiểm tra. Benchmark một máy chưa chứng minh chịu lỗi hay mở rộng nhiều node. Các hướng phát triển chưa hoàn thành.

→ Chuyển slide: Em xin kết thúc phần trình bày.

Slide 20/20 Trân trọng cảm ơn quý thầy cô đã lắng nghe

10 giây hết slide lúc 15:00



Nhìn vào
Hướng về quý thầy cô khi dòng mời đặt câu hỏi xuất hiện trên slide.

Đọc

Em xin trân trọng cảm ơn quý thầy cô đã lắng nghe. Em xin nhận câu hỏi và góp ý của quý thầy cô.

Nói thêm nếu còn thời gian hoặc được hỏi
Mở lại slide có bằng chứng phù hợp khi trả lời; không chạy lại huấn luyện.

→ Chuyển slide: Dừng tại slide này để trao đổi với quý thầy cô.

Câu hỏi phản biện

Trả lời ngắn trước, chỉ thêm chi tiết khi được hỏi tiếp. Nếu cần hình minh họa, quay lại slide ghi trong ngoặc.

Hỏi: Vì sao chọn EWMA để khử nhiễu? (slide 13)

Trả lời: EWMA chỉ dùng hiện tại và quá khứ, không dùng tương lai, có ít tham số và giảm dao động trong khi giữ xu hướng chậm. Span 10 được cố định trước, không chỉnh theo kết quả test.

Hỏi: Không có nhãn bất thường thì kiểm chứng thế nào? (slide 15)

Trả lời: Em so mỗi động cơ với 30 cycle đầu của chính nó và đo thời gian báo trước khi hỏng. Chưa đo được báo động giả vì mọi động cơ train đều đi đến hỏng và không có nhãn bắt đầu suy giảm. Trên test, mức theo dõi còn rộng, dù bao gồm các động cơ cần bảo trì bị bỏ sót ở mức hai.

Hỏi: Vì sao chọn 3 sigma và 5 cycle liên tiếp? (slide 15)

Trả lời: Đây là quy tắc cố định trước: độ lệch chuẩn đo mức dao động quanh tham chiếu; yêu cầu liên tiếp hạn chế cảnh báo do một giá trị ngoại lai. Em kiểm tra lưới 2, 3, 4 sigma kết hợp 3, 5, 10 cycle, tổng cộng 9 cấu hình. Tất cả phát hiện 100/100 động cơ, thời gian báo trước nhỏ nhất ít nhất 50 cycle. Với 2 sigma, 3 cycle, trung vị là 136 cycle nhưng có 17 động cơ báo ngay sau đoạn tham chiếu. Với 4 sigma, 10 cycle, không có trường hợp đó, nhưng giảm khoảng 22 cycle báo trước. Cấu hình chính có một trường hợp báo quá sớm, trung vị 121 cycle. Khả năng phát hiện ổn định trong lưới đã thử; thời điểm và mức báo sớm vẫn phụ thuộc cấu hình.

Hỏi: Vì sao chia GroupKFold theo động cơ? (slide 16)

Trả lời: Hai window liền kề trùng 29/30 cycle. Chia ngẫu nhiên theo dòng sẽ đưa những mẫu gần giống nhau vào cả train và validation, làm đánh giá quá thuận lợi. Vì vậy em giữ mọi window của một động cơ trong cùng một fold.

Hỏi: Có rò rỉ dữ liệu không? (slide 9, 11, 15, 16)

Trả lời: Window, EWMA và quy tắc bất thường chỉ dùng hiện tại và quá khứ; mọi window của một động cơ nằm cùng fold. Health Index chỉ học trên train. Nhãn test chỉ đọc sau khi lưu dự đoán và lịch bảo trì.

Hỏi: Vì sao 49% cảnh báo bị trễ? (slide 17)

Trả lời: Mô hình có xu hướng dự đoán dư RUL, đặc biệt với động cơ suy giảm nhanh, nên dự đoán chạm ngưỡng 30 muộn hơn mốc mong muốn. Ngưỡng 40 giảm tỷ lệ trễ trên train OOF xuống 6%, đổi lại báo sớm nhiều hơn. Đây chưa phải ngưỡng tối ưu về chi phí.

Hỏi: Mức theo dõi tăng cường có quá rộng không? (slide 18)

Trả lời: Có. Trong 59 động cơ thuộc mức này, 41 còn trên 60 cycle. Mức này phục vụ tăng tần suất kiểm tra, không có nghĩa tất cả cần bảo trì ngay.

Hỏi: Có lịch bảo trì sau thời điểm hỏng thật không? (slide 18)

Trả lời: Có 12/100 động cơ có cycle bảo trì dự kiến muộn hơn lúc hỏng 4–27 cycle. Tất cả còn ít nhất 77 cycle theo nhãn thật, thuộc vùng mô hình dự đoán dư RUL. Cách vận hành dự kiến là cập nhật lịch khi có dữ liệu mới, nhưng chưa kiểm chứng sẽ sửa kịp những trường hợp này. Vì vậy không nên chốt lịch xa từ một dự đoán. Kết quả 21/21 chỉ áp dụng cho các động cơ được chỉ định bảo trì ngay.

Hỏi: Dữ liệu chỉ 5,7 MB, bỏ Hadoop và Spark thì lịch bảo trì có đổi không? (slide 12)

Trả lời: Với FD001 hiện tại, lịch không đổi. Khi nhân bản 100 lần, Pandas mất khoảng 18 phút, Spark khoảng 3 phút; ở 200 lần, Pandas chưa xong sau 30 phút, Spark khoảng 6,5 phút. Đây là thời gian tính toán, chưa gồm ghi file. Đóng góp dữ liệu lớn là quy trình phân lớp và kiểm chứng phép tính giữa các công cụ, cùng bằng chứng benchmark ở quy mô đã đo; chưa chứng minh mở rộng nhiều máy.

Hỏi: Hadoop nằm ở đâu trong bài? (slide 8)

Trả lời: Hadoop 3.4.3 chạy trong Docker, có NameNode, DataNode cho HDFS và ResourceManager, NodeManager cho YARN. File raw được đưa lên HDFS. MapReduce qua Hadoop Streaming tính số cycle, cycle đầu và cycle cuối của từng động cơ. Spark đọc HDFS, ghi Silver Parquet rồi tổng hợp. Cả hai khớp 200/200 động cơ với Pandas. Map sinh khóa động cơ và dữ liệu cycle; reduce tổng hợp các giá trị cùng khóa. Block mặc định 128 MB; dung lượng nhỏ không có nghĩa toàn bộ các file HDFS chỉ chiếm một block.

Hỏi: Vì sao dùng Parquet thay vì chỉ giữ text? (slide 7)

Trả lời: Bronze vẫn giữ text gốc. Parquet lưu theo cột, phù hợp truy vấn một nhóm sensor và giữ kiểu dữ liệu rõ ràng. Silver dùng chung cho Pandas và Spark; bảng tổng hợp nhỏ còn lưu bằng SQLite để truy vấn quan hệ. Việc chọn định dạng không tự chứng minh hệ thống đã đạt quy mô lớn.

Hỏi: HDFS một node chứng minh được gì? (slide 8)

Trả lời: Chứng minh file đã được lưu và đọc trên HDFS, job chạy qua YARN, kết quả tổng hợp đúng trong phép đối chiếu. Trạng thái HEALTHY phản ánh kiểm tra lưu trữ trong cấu hình hiện tại. Một node không chứng minh chịu lỗi giữa các máy hay tăng tốc nhờ phân tán.

Hỏi: Vì sao Spark nhanh hơn ở 100×? (slide 12)

Trả lời: Spark local[4] sử dụng bốn luồng, trong khi Pandas trong phép đo chạy một luồng. Khi dữ liệu lớn, lợi ích xử lý song song có thể bù chi phí khởi tạo và điều phối. Kết quả 5,5 lần áp dụng cho phép đo này, không tách riêng được đóng góp của từng yếu tố và không phải cam kết cho mọi máy. Bước ghi đầy đủ vẫn gặp giới hạn khi gom dữ liệu về driver.

Hỏi: Vì sao cần khóa ba thành phần? (slide 6, 10, 11)

Trả lời: Với bảng quan sát và đặc trưng, split phân biệt train/test, engine_id phân biệt động cơ, cycle phân biệt thời điểm. Thiếu split có thể trộn hai tập; thiếu cycle khi ghép nhãn có thể nhân dòng. Bảng tổng hợp theo động cơ chỉ cần (split, engine_id). Ví dụ ghép sai của động cơ train 1 tạo 31.296 dòng thay vì 163 dòng.

Hỏi: Sao không dùng LSTM hay Transformer? (slide 19)

Trả lời: Bài tập trung vào pipeline dữ liệu, nên em chọn mô hình cây trên đặc trưng thống kê để kiểm tra được từng bước. Mô hình sâu cần được so trên cùng cách chia theo động cơ; em chưa làm thí nghiệm đó nên chưa kết luận mô hình sâu tốt hơn hay kém hơn.

Hỏi: Kết quả có dùng ngay cho động cơ thật không? (slide 19)

Trả lời: Chưa. C-MAPSS là dữ liệu mô phỏng, không có chi phí bảo trì hay lịch sử sửa chữa. Mô hình cũng chỉ được huấn luyện và đánh giá trên FD001, tức một điều kiện vận hành và một dạng lỗi.

Hỏi: Nếu chưa có căn cứ trả lời

Trả lời: Dạ, phần này em chưa kiểm tra trong bài; em xin ghi nhận để bổ sung. Không suy đoán số liệu hoặc khẳng định kết quả chưa đo.

Số liệu tra nhanh

Nội dung	Số liệu
Dữ liệu FD001	100 động cơ train, 100 test; 20.631 dòng train, 13.096 dòng test; 21 sensor
Pandas vs PySpark	Sai khác 0. Tính toán (không ghi file): 100×: Pandas 1.075 giây, Spark 196 giây (nhanh hơn 5,5 lần); 200×: Pandas không xong trong 30 phút, Spark 392 giây
Hadoop	3.4.3 một node trong Docker; MapReduce trên YARN (3 map, 1 reduce) và Spark đọc HDFS đều khớp 200/200; fsck HEALTHY
Thu thập	Tải từ data.nasa.gov, MD5 gói khớp, 12/12 file đúng SHA-256
Khử nhiễu EWMA	Span 10, $\alpha \approx 0{,}18$; giảm 83,6–86,4% dao động, trung vị 86,3%
Window	30 cycle, 21 × 6 = 126 đặc trưng; 17.731 window train, 10.196 window test
Xu hướng	15/21 sensor thay đổi có ý nghĩa; 12 sensor đổi cùng hướng ở 100/100 động cơ, ví dụ sensor 4, 7, 11, 12, 20
Bất thường	100/100 động cơ train; trung vị 121 cycle trước khi hỏng (72,5 nếu không khử nhiễu); đã qua khoảng 39% vòng đời
Độ nhạy bất thường	9 cấu hình (2/3/4σ × 3/5/10 cycle) đều 100/100; lead time nhỏ nhất $\geq$ 50; 2σ–3 cycle: trung vị 136, 17 báo sớm; 4σ–10 cycle: 98,5, 0 báo sớm
Mô hình	HistGradientBoosting, GroupKFold 5 fold; MAE CV 20,21 (Ridge 23,99)
Kết quả test	MAE 15,35 (CI 95%: 12,73–18,09); RMSE 20,54; NASA 1.513; MAE 4,38 khi RUL thật $\leq$ 30
Cảnh báo ngưỡng 30 (OOF train)	44 trước, 7 đúng lúc, 49 muộn, 0 bỏ sót; báo trước trung vị 30 cycle
Cảnh báo ngưỡng 40	Kịp thời 94% (ngưỡng 30: 51%); trễ 6%
Lịch bảo trì	21 cần bảo trì, 59 theo dõi, 20 bình thường; 21/21 đúng; 14 đến hạn trong 25 cycle tới; 12 động cơ có lịch muộn hơn lúc hỏng 4–27 cycle (RUL thật đều $\geq$ 77)
Ví dụ động cơ 1	Bất thường cycle 84; cảnh báo RUL cycle 162; hỏng cycle 192
Ví dụ động cơ 90	Cycle 146; RUL dự đoán 31,2; bảo trì cycle 147; bất thường từ cycle 67
Chuyển miền	MAE 47,39–60,32 (huấn luyện trên chính subset: 22,36–28,33); tăng 1,7–2,7 lần